

Closing the Loop: Spec-Grounded, Self-Verifying AI Authoring of X3D 4.0 Scenes in the Browser

Technical overview of the *X3D Copilot* prototype

Alberto Jaspe-Villanueva, King Abdullah University of Science and Technology (KAUST) alberto.jaspe@kaust.edu.sa

Prototype: <https://albertojaspe.net/x3d-copilot/> Source (MIT): <https://github.com/ajaspe/x3d-copilot>
Web3D 2026 – AI & Web3D Innovation Competition – category: AI-Driven Content Generation (also relevant: Open Innovation)

To try the AI live: open **Settings** in the prototype and paste your own Gemini (free tier at aistudio.google.com/apikey) or Claude API key; the key stays in your browser and is sent only to the provider. Editing, validation, the gizmo and import/export work without a key.

Summary

Large language models can draft X3D quickly, but on their own their output is unreliable: invented field names, misplaced nodes, broken ROUTEs, mixed-up versions, and no way to see what they produced. This prototype takes a different route from “a bigger model or a longer prompt”: it **grounds the model in the standard itself** (the ISO/IEC 19775-1 X3D Unified Object Model, compiled into a queryable database) and **closes the loop with verification the model cannot bypass** (the official X3D 4.0 XML Schema, a semantic linter derived from the object model, the X_ITE runtime, and a screenshot the model inspects). The result is AI-generated 3D content that is *standards-conformant by construction*, produced inside a complete browser-based X3D editor with live rendering, direct manipulation and multi-format import. Everything runs client-side as a static site; the only network traffic is the model call, made with the user’s own key.

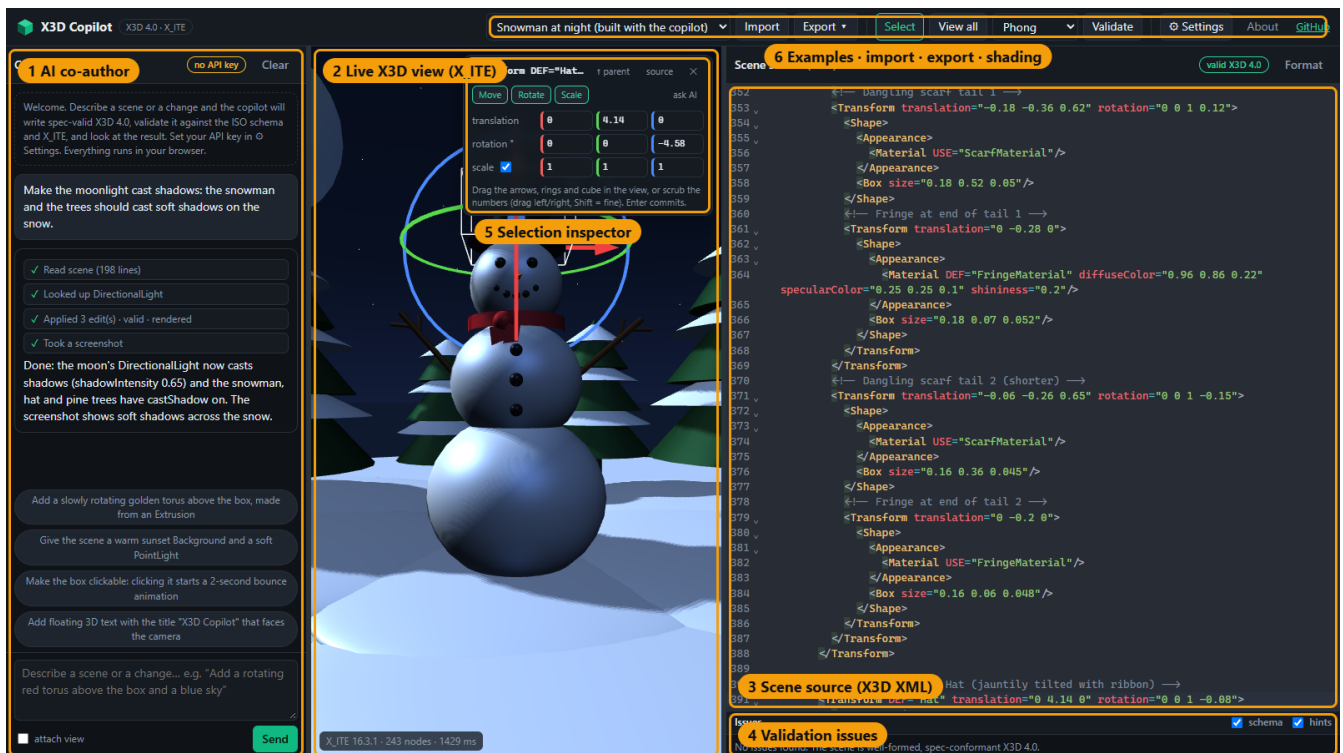


Figure 1: The interface, with the copilot-authored “snowman” example loaded and its hat selected. (1) **AI co-author:** the conversation with the model; every tool call is shown as a step (read scene, look up a node in the spec, apply edits, validate, screenshot) that the user can expand to read the exact validation report the model received. (2) **Live X3D view:** X_ITE 16 renders the scene as the source changes; clicking geometry selects its **Transform**, and the gizmo (arrows, rings, cube) is itself built from X3D **PlaneSensor/CylinderSensor** nodes injected at runtime. (3) **Scene source:** a CodeMirror editor with autocompletion generated from the X3D object model (nodes, allowed children, fields with types and defaults) and inline diagnostics. (4) **Validation issues:** findings from the three validators, graded and attributed (xml/lint/schema/runtime), click to jump to the line. (5) **Selection inspector:** translation, rotation and scale of the selected node, scrubbable; edits are written back to the source as minimal attribute changes, and the selection is passed to the model as context. (6) **Toolbar:** examples, import of glTF/OBJ/STL/PLY/VRML, export to X3D XML/Classic VRML/JSON/PNG, shading modes, model settings.

How the AI is integrated

[AI Integration 20%] Models. Google Gemini (3.8 Flash by default; 3.1 Pro and 2.5 selectable) and Anthropic Claude (Opus 5, Sonnet 5, Haiku 4.5), through their official JavaScript SDKs, directly from the browser. A small provider interface isolates each SDK in an adapter of about 150 lines, so the architecture is model-agnostic. Streaming, native function calling, images returned inside tool results (so the model can see its render) and effort/thinking controls are used where the provider supports them.

The loop. The model never writes to the scene directly. A provider-neutral agent loop sends the system prompt,

the tool definitions and the conversation; executes the returned tool calls locally; appends their results; and repeats (up to 24 rounds per user turn). Seven tools:

- **get_scene**, **edit_scene** (exact, unique text replacements or anchored insertions; ambiguous edits are rejected), **replace_scene**, **validate_scene** – each returns the full graded issue list with line numbers plus the X_ITE render status, so the model repairs its own mistakes before answering;
- **lookup_node**, **search_nodes** – read the ISO object model: every field with type, accessType, default, accepted node types and containerField, the component and the specification URL; nodes the running X_ITE build does not implement are flagged;
- **screenshot** – returns the current frame as an image so the model can check framing, lighting and visibility.

The system prompt is deliberately short: it fixes the workflow (read before edit, prefer patches, fix errors before answering, look up when unsure, screenshot when appearance matters) and a compact X3D 4.0 authoring guide. Correctness comes from the tools, not from prompt text. A per-message note carries the user’s current selection, so deictic requests (“make *this* twice as big”) resolve to an exact node.

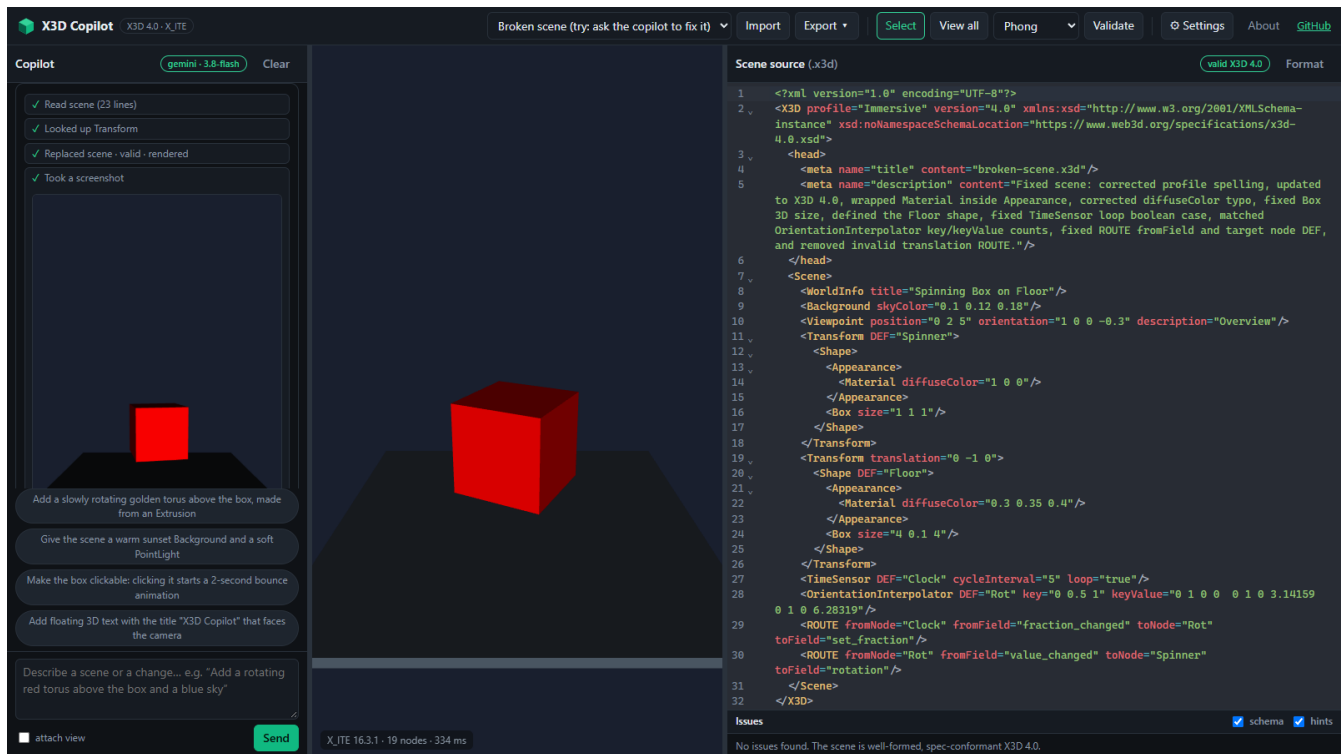


Figure 2: The loop in action on the shipped “broken scene” example (eleven planted errors: misspelled profile, stale version, a **Material** outside its **Appearance**, a wrong field name, a 2-component **size**, an undefined USE, a capitalised boolean, mismatched interpolator keys, and three bad ROUTEs). Left: the model’s steps, with the validation report it received after replacing the scene expanded, and the screenshot it took to verify the render; centre: the repaired scene; right: the corrected source, now reported as valid X3D 4.0 by all validators.

Verification stack

[Technical Excellence 25%]

1. **XSD:** the official `x3d-4.0.xsd` (with the Web3D extension schemas and the W3C XML-Signature schema it imports) is vendored and compiled once by libxml2 running in WebAssembly; validating a typical scene takes a few milliseconds.
2. **Semantic linter:** about thirty rules the schema cannot express, all driven by the object-model database: duplicate DEF, undefined USE, USE of a different node type, ROUTE to an unknown node or field, ROUTE from a non-output or to a non-input field, ROUTE type mismatch, a node in a parent field that does not accept it, several children in an SFNode field, attribute arity/type/range checks, zero rotation axes, interpolator key/keyValue consistency, empty groups, Shape without geometry, missing Viewpoint, unknown nodes and fields with did-you-mean suggestions, and nodes unsupported by the running X_ITE build.
3. **Runtime:** X_ITE parse errors and console warnings (missing textures, bad URLs, unsupported profiles) are captured during load and reported.

Schema messages that duplicate a linter finding on the same line are demoted so the model is not double-penalised. The three layers are independent modules with a shared issue format, and the spec database is regenerated from the X3DUOM by a script.

Web3D frameworks and standards

X3D 4.0 (ISO/IEC 19775-1:2023) and its XML encoding (ISO/IEC 19776-1); the X3D Unified Object Model 4.0 (Web3D Consortium), compiled at build time into a database of 260 concrete nodes, 78 abstract types and 17 statements;

X_ITE 16.3 as rendering and runtime engine, including its glTF/OBJ/STL/PLY/SVG loaders and its XML/Classic VRML/JSON encoders (imports are normalised to X3D 4.0); CodeMirror 6 for the editor, with completions generated from the object model. All scenes the tool produces are plain, portable `.x3d` files; the gizmo, the selection sensors and everything else the tool adds at runtime live only in the X_ITE scene graph and never enter the user's file.

User experience

[User Experience 15%] Zero installation: open a URL. Natural-language authoring with visible, expandable tool steps, so the user always knows what the model read, changed and verified. Direct manipulation with an X3D-native gizmo and a numeric inspector, synchronised two-way with the source; the text cursor selects nodes in the view and vice versa. Errors are phrased as actions (“<Material> goes inside <Appearance>”), with did-you-mean suggestions and jump-to-line. Eight example scenes, including one authored entirely through the copilot in four sentences (the snowman above) and a deliberately broken one for trying the repair loop.

Why we think it is novel

[Innovation & Originality 30%] To our knowledge this is the first X3D authoring tool in which an LLM (a) reads node and field definitions from the ISO object model through a tool instead of relying on training memory, (b) can only act through validated edits whose reports are fed back to it, and (c) inspects a screenshot of its own render before answering; and the first browser X3D editor whose transform gizmo is expressed in X3D sensor nodes. None of the individual ingredients is new; the contribution is the combination, and the fact that it makes AI-generated X3D conformant by construction rather than by luck. We have measured this informally rather than with a benchmark: in our recorded sessions the model repairs the eleven-error scene in one turn and builds complete animated, interactive scenes (nested orbits, sensors, PBR materials, shadows, click behaviours) from short prompts, and we ship those scenes as examples.

Impact and scalability

[Impact & Scalability 10%] For the Web3D community the tool lowers the entry barrier to X3D to a sentence in a chat box while *raising* conformance, which helps education, rapid prototyping and the health of the standard. The pattern (machine-readable specification + validators as tools + visual check) is not specific to X3D: the object-model database is generated by a script, and the same architecture applies to glTF, USD or CityGML with a different schema and object model. Scalability is trivial in the deployment sense, since the tool is a static site and the compute is the user's browser plus their own model account. Everything is open source under the MIT license, covered by 56 automated tests (schema fixtures, one test per linter rule, edit-tool patching, gizmo/source-map maths, and a check that every shipped example is lint-clean and schema-valid), with continuous integration on GitHub.

Limitations

Selection targets **Transform** nodes only; ProtoInstance internals are not selectable. The screenshot check is qualitative and the model's judgement of aesthetics varies between runs. Long generations depend on the provider's speed and quotas. Planned next steps: HAnim-aware tools (joint lookup, motion import), glTF export through X_ITE, a retrieval tool over the X3D examples archive, and a quantitative evaluation of first-pass validity with and without the spec tools.